Newton School of Technology

System Design & Software Engineering Project

Deepa's Vision: The Vedic Astrology Intelligence Platform

Prepared By:

Vetriselvan R	2401010501
Hrishi Dongre	2401010191
Vedant Madne	2401010499
Ishan Goyal	2401010193
Tattva Rajput	2401010478

Contents

1 Project Introduction **2**
- 1.1 Problem Statement . . . 2
- 1.2 Project Objectives . . . 2
- 1.3 Technical Overview . . . 2

2 System Design & Architecture **3**
- 2.1 N-Tiered Layered Architecture . . . 3
- 2.2 Importance of SD Concepts in this Project . . . 3

3 Object-Oriented Programming (OOP) Deep Dive **3**
- 3.1 Core Concepts . . . 3
  - 3.1.1 Classes & Objects . . . 3
  - 3.1.2 Constructors & Dependency Injection . . . 4
  - 3.1.3 Invariants . . . 4
- 3.2 OOP Mechanisms . . . 4
  - 3.2.1 Encapsulation (Access Control) . . . 4
  - 3.2.2 Abstraction & Interfaces . . . 4
  - 3.2.3 Inheritance & Polymorphism . . . 4
- 3.3 Object Relationships . . . 5
  - 3.3.1 Composition . . . 5
  - 3.3.2 DTOs & Domain Objects . . . 5

4 Unified Modelling Language (UML) **5**
- 4.1 Structural: Class Diagram . . . 5
- 4.2 Behavioral: Sequence Diagram . . . 5

5 Entity Relationship Diagram (ERD) **5**
- 5.1 Entities and Attributes . . . 5
- 5.2 Mapping Cardinalities . . . 6

6 Design Patterns Implementation **6**
- 6.1 Creational: Singleton & Factory . . . 6
- 6.2 Structural: Adapter & Composite . . . 6
- 6.3 Behavioral: Strategy & Observer . . . 6

7 SOLID Principles Analysis **6**

8 Conclusion & Future Scope **7**
- 8.1 Future Enhancements . . . 7

1 Project Introduction

1.1 Problem Statement

Vedic Astrology (Jyotish) is a complex science involving high-precision astronomical calculations. Most modern consumers lack access to transparent, data-driven astrology tools, often relying on generic horoscopes that do not account for precise birth coordinates or specific planetary alignments like *Doshas*. Furthermore, for developers, building such platforms often leads to "spaghetti code" due to the high number of external API dependencies and varying data formats. There was a clear need for a structured, scalable, and secure platform that bridges the gap between ancient science and modern software engineering.

1.2 Project Objectives

Deepa's Vision was conceived with three primary goals:

1. To provide a high-fidelity Vedic identity platform powered by real-time astronomical data.
2. To implement a secure, tiered architecture that protects user data while maintaining high performance.
3. To serve as a showcase for advanced System Design (SD) and Software Engineering (SE) principles in a real-world full-stack application.

1.3 Technical Overview

The platform is a full-stack solution utilizing:

- **Frontend:** Next.js 14 with App Router, utilizing React's component-based architecture for a modular UI.
- **Backend:** Node.js with Express and TypeScript, structured into a 3-tier MVC-S (Model-View-Controller-Service) architecture.
- **Database:** MongoDB Atlas for persistence, with specialized TTL (Time-To-Live) indexes for automated cache invalidation.
- **Integration:** VedicAstro API for astronomical computations (Natal Charts, Dosha Severity).

2 System Design & Architecture

2.1 N-Tiered Layered Architecture

We implemented a strict separation of concerns through a layered approach:

1. **Route Layer:** Entry points that define API contracts.
2. **Middleware Layer:** Handles cross-cutting concerns like JWT verification, rate limiting, and request validation.
3. **Controller Layer:** Orchestrates the request/response cycle. It does *not* contain business logic but delegates to services.
4. **Service Layer:** The core of the application. It handles external API communication, business rules, and data formatting.
5. **Model/Data Layer:** Manages interactions with MongoDB using Mongoose, enforcing schema-level invariants.

2.2 Importance of SD Concepts in this Project

In a project with multiple external dependencies (like the VedicAstro API), the system would quickly become unmanageable without SD principles. For example:

- **Abstraction** allowed us to switch from a mock API to the real production API during development with minimal code changes.
- **Design Patterns** like *Strategy* allowed us to handle 5+ different Dosha types using a single service method, rather than a 50-line `switch` statement.
- **SOLID Principles** ensured that as we added new features (like Pitra Dosh or Nadi Dosh), we didn't break existing functionality (Manglik Dosh).

3 Object-Oriented Programming (OOP) Deep Dive

3.1 Core Concepts

3.1.1 Classes & Objects

Every logical unit in the backend is a Class. This allows for better organization and state management. For instance, the `AstroService` class maintains its own Axios instance, ensuring that headers and base URLs are encapsulated and not leaked.

3.1.2 Constructors & Dependency Injection

We avoided hard-coding dependencies. Instead, we used constructor-based injection.

```
// Dependency Inversion at work
export class DoshaController extends BaseController {
  constructor(private astroService: IAstroService) {
    super(); // Inherit BaseController functionality
  }
}
```

3.1.3 Invariants

Invariants are rules that must always be true for an object to be valid. In our `UserProfile` model, we enforce that `timeOfBirth` must match a specific `HH:mm` regex, and coordinates must be valid numbers. This prevents "dirty data" from reaching the business logic layer.

3.2 OOP Mechanisms

3.2.1 Encapsulation (Access Control)

We use TypeScript's access modifiers (`private`, `protected`, `public`) to define clear boundaries.

- **Private:** Internal helper methods like `calculateDegrees()` in services are hidden from controllers.
- **Protected:** Methods in `BaseController` (like `ok()`) are available to child controllers but hidden from the rest of the app.

3.2.2 Abstraction & Interfaces

We define blueprints using interfaces. The `IAstroService` interface ensures that any service implementation (regardless of the external provider) must provide `fetchBirthChart()` and `fetchDosha()` methods.

3.2.3 Inheritance & Polymorphism

All controllers inherit from `BaseController`, inheriting the `asyncHandler` which handles all try/catch logic automatically. Polymorphism allows the router to treat all controllers as a standard `Handler` type while executing their specific logic.

3.3 Object Relationships

3.3.1 Composition

The `DoshaController` is *composed* of an `AstroService` and a `CacheService`. This "Has-A" relationship is preferred over "Is-A" (Inheritance) because it makes the controller more flexible and easier to test.

3.3.2 DTOs & Domain Objects

We use `VedicParams` as a DTO to move user birth data from the controller to the service layer. This ensures that the service layer only receives exactly what it needs to perform calculations.

4 Unified Modelling Language (UML)

4.1 Structural: Class Diagram

Our Class Diagram shows a clear hierarchy. The `BaseController` acts as a root for `AuthController`, `ProfileController`, and `DoshaController`. Note the **Dependency Inversion** where controllers depend on the `IAstroService` interface rather than the concrete `AstroService` class.

4.2 Behavioral: Sequence Diagram

The Sequence Diagram reveals the "Request-Cache-API" flow:

1. **User** submits birth data via the Next.js UI.
2. **Controller** receives the request and checks the **MongoDB Cache**.
3. If **Cache Hit**: Returns the stored report immediately (Response time: <50ms).
4. If **Cache Miss**: Calls the **AstroService Strategy**, which invokes the **VedicAstro API**.
5. **Service** formats the response, saves it to the cache, and returns it.

5 Entity Relationship Diagram (ERD)

5.1 Entities and Attributes

- **USER:** Stores authentication data. Sensitive fields (`password`) are hidden from query results by default.

- **USER_PROFILE:** Linked to `USER` via a **Unique Foreign Key** (`userId`). Contains nested objects for birth location.
- **DOSHA_REPORT:** Contains the analysis result. It includes an `expiresAt` field for TTL caching.

5.2 Mapping Cardinalities

- **1:1 (User to Profile):** Enforces that each user manages only their own astrological identity.
- **1:N (User to Charts):** Allows users to generate and save multiple charts over time.

6 Design Patterns Implementation

6.1 Creational: Singleton & Factory

The `AstroService` is initialized as a Singleton. This is crucial because it maintains a connection pool via Axios; creating a new instance for every request would lead to memory leaks and socket exhaustion.

6.2 Structural: Adapter & Composite

- **Adapter Pattern:** The `AstroService` "adapts" the external API's raw response (which might have inconsistent naming) into our application's `IDoshaReport` format.
- **Composite Pattern:** Our frontend's `ReportCard` component is a composite of `SeverityBadge`, `RemedyList`, and `ChartGrid` components.

6.3 Behavioral: Strategy & Observer

- **Strategy Pattern:** The `DOSHA_ENDPOINT_MAP` allows the service to switch endpoints dynamically based on the `doshaType` string.
- **Observer Pattern:** Mongoose's `toJSON` method "observes" the model serialization. It automatically removes `password` and `tokens`, acting as a security sentinel.

7 SOLID Principles Analysis

- **SRP:** The `AuthMiddleware` has one job: verify the token. It does not handle database logic or response formatting.

- **OCP:** Adding "Saturn Dosh" would only require adding a new key to the `DOSHA_ENDPOINT_MAP`. The `fetchOtherdosha` function remains untouched.
- **LSP:** We can replace the `AstroService` with a `MockAstroService` in our unit tests, and the `DoshaController` will work perfectly because both implement `IAstroService`.
- **ISP:** We split `IAstroService` and `ICacheService`. A developer working on the profile page doesn't need to see methods related to Redis caching.
- **DIP:** High-level modules (Controllers) never import low-level modules (Axios/Vedic API). They interact through Service abstractions.

8 Conclusion & Future Scope

The development of *Deepa's Vision* was an exercise in building scalable software for a niche but data-heavy domain. By grounding the project in robust System Design principles, we created a platform that is not only functional but also academically sound.

8.1 Future Enhancements

- **Microservices Transition:** As the user base grows, we plan to decouple the `AstroService` into a separate microservice to handle high-concurrency API calls.
- **Offline Support:** Implementing a Progressive Web App (PWA) strategy to allow users to view their saved charts without an internet connection.